Методические указания к лабораторной работе №4

Делегаты и события. Механизм подписки на события. Класс web-клиент, http get-запрос.

В данной работе на практике закрепляются функциональные аспекты языка C#: делегаты и события. Событийные модели широко распространены в языках программирования прежде всего для организации механизма уведомлений объектов-подписчиков о факте окончания какой-либо операции объекта-издателя. В данной лабораторной работе событийная модель позволяет логически отделить алгоритм поиска информации на веб-ресурсе от алгоритма обработки найденных данных.

Идея реализации сканера веб-ресурса в том, чтобы, во-первых, реализовать и отладить алгоритм загрузки и анализа веб-страниц, причем рекурсивно пробегая по всем страницам сайта, инициируя события, а во-вторых, предать целевые данные обработчику события для отображения информации на консоли, записи в файл и др.

Веб-страница – это текст на языке HTML, поэтому ее можно анализировать любыми средствам анализа текста, например, регулярными выражениями. Альтернативный способ – превратить HTML-страницу в дерево объектов-элементов, т.е. DOM-модель, используя различные сторонние библиотеки по работе с HTML. Остановимся на первом варианте.

Загрузим страницу и посмотрим, что у нее внутри. Удобно использовать класс WebClient, который позволяет загружать как текст, так и бинарный контент по заданному URI (адресной строке).

```
WebClient client = new WebClient();
string page = client.DownloadString(new Uri("https://www.susu.ru/"));

File.WriteAllText(@"c:\Temp\page.txt", page);
```

Сохраненную страницу можно открыть в блокноте.

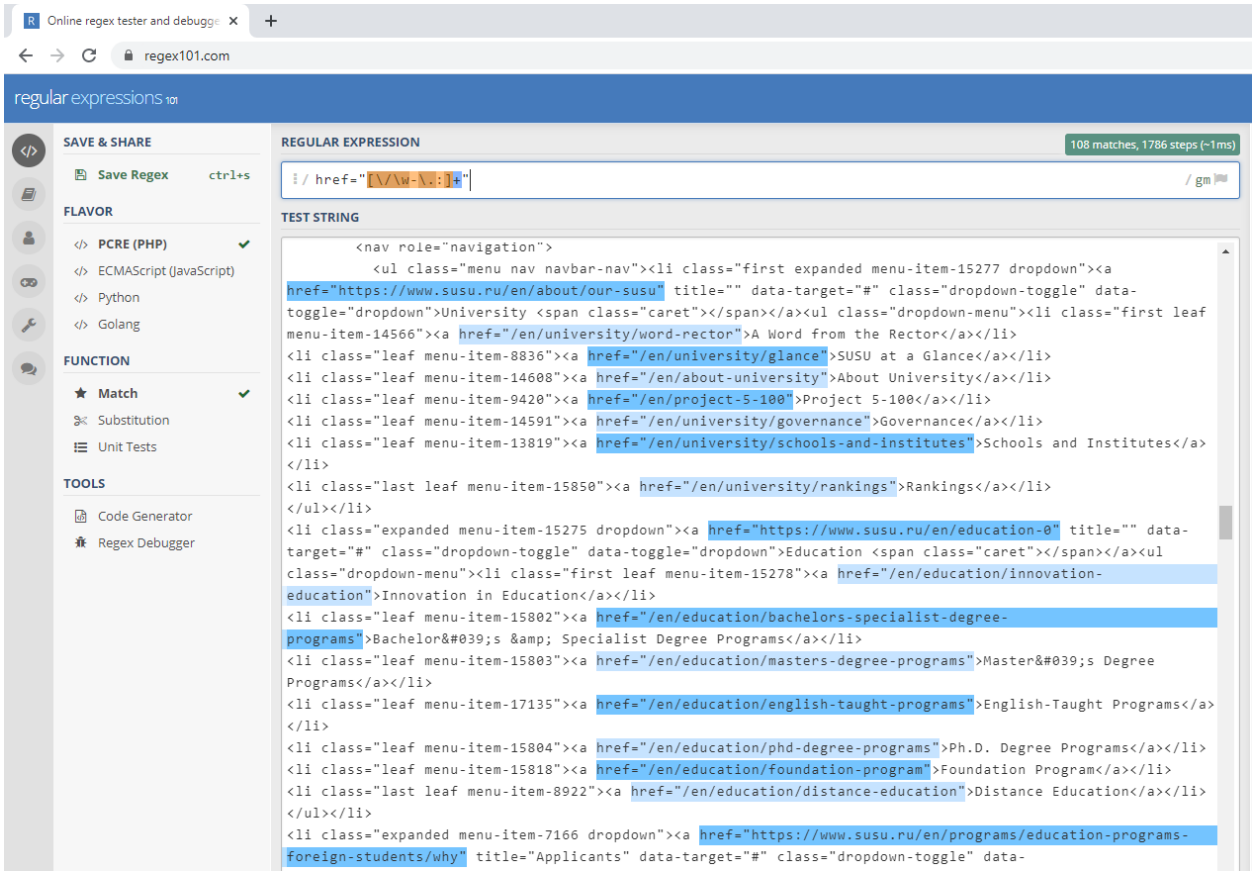
```
</div><input type="hidden" name="form_build_id" value="form-
BNylVHbx1bBVmTZAM_iytfG1Dzcsg8x1VFsD5zkiih8" />
<input type="hidden" name="form_id" value="sphinxdrupal_search_box" />
</div></form>
</section>
<section id="block-locale-language" class="block block-locale clearfix">


  <ul class="language-switcher-locale-url"><li class="ru first"><a href="/ru" class="
  language-link" xml:lang="ru">Р СѓСЃСЃРєРёР№</a></li>
<li class="en active"><a href="/en" class="language-link active" xml:lang="en">English</a><
/li>
<li class="ar"><a href="/ar" class="language-link" xml:lang="ar">Ш§Щ„,,ШЊШ±ШЁЩ‰Ш©</a></li>
<li class="zh-hans last"><a href="/zh" class="language-link" xml:lang="zh-hans">
з®ЂдЅ“дё-ж-‡</a></li>
</ul>
</section>
    <a href="https://white.susu.ru/en" id="blind">Version for visually impaired</a>
  </div>
      </div>
```

Прежде всего обратим внимание на ссылки внутри страницы, они нужны для организации загрузки тех страниц, на которые ссылается эта первичная или стартовая страница. Ссылка – это тег **<a />** с атрибутом **href**. Ссылки бывают относительными, т.е. ведущими на страницы этого же

сайта, и абсолютными, ведущими на внешние ресурсы. Относительные ссылки не имеют доменного имени сайте и начинаются со слеша.

Регулярные выражения – это строковые шаблоны для поиска информации в тексте. Удобный сайт для отладки https://regex101.com. Загрузим текст страницы и напишем шаблон поиска ссылок внутри страницы.



Шаблон получился: **href="[\/\w-\.:]+"**, т.е. ищем строку, начинающуюся с href=", оканчивающуюся ", а между кавычками символы, встречающиеся один или более раз, это “/”, “-“, “.”, “:”, любая латинская буква, цифра, пробел “\w”.

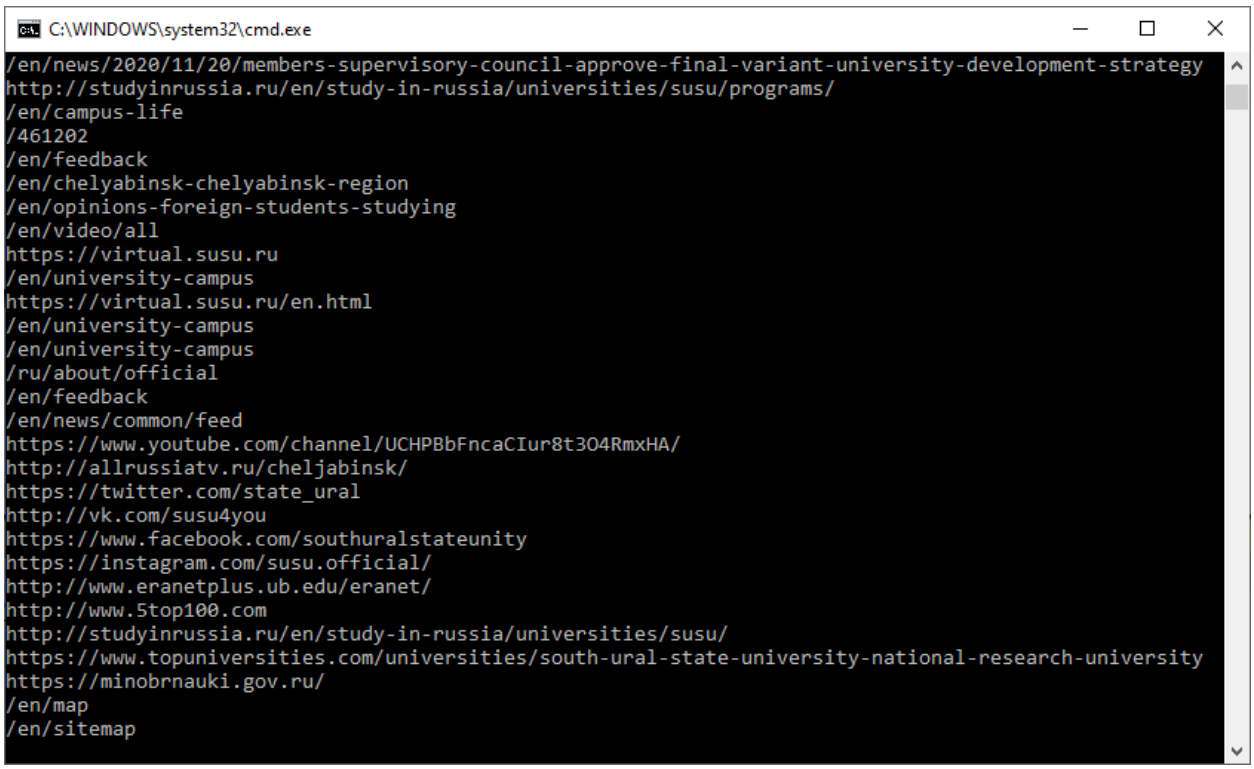
Следующий код анализирует текст страницы, находит в ней подстроки по регулярному выражению.

```
var hrefs = Regex.Matches(page, @"href=""[\/\w-\.:]+""").Cast<Match>()
                 .Select(r => r.Value.Replace("href=", "").Trim('"'));

foreach (var href in hrefs)
    Console.WriteLine(href);
```

Метод Regex.Matches() выдает коллекцию подстрок, найденных по шаблону регулярного выражения, LINQ-метод Cast<>() преобразует коллекцию к IEnumerable<Match>, чтобы затем применить LINQ-метод Select() для преобразования каждой найденной подстроки (тип данных Match) к типу string, убрать из строки лишние кавычки и href.

Посмотрим на результат.



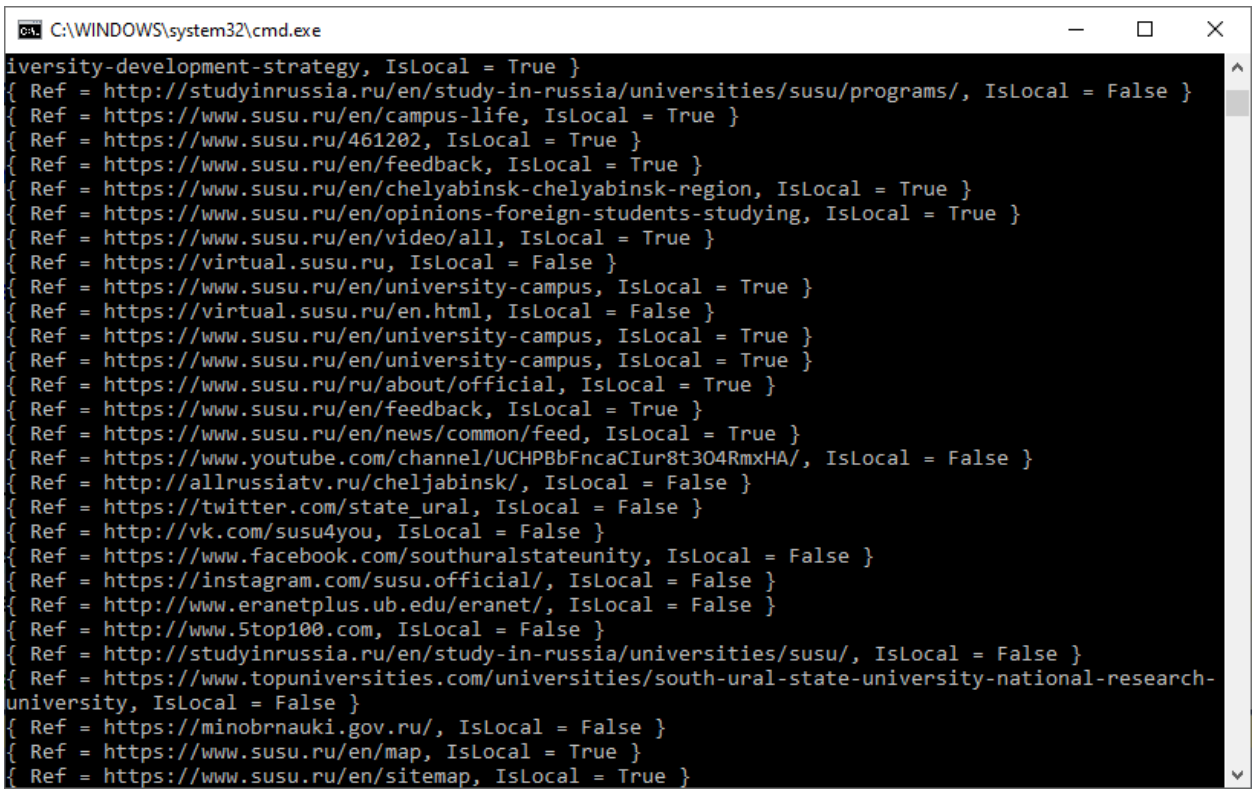
Для удобства дальнейшей работы можно явно найти локальные ссылки, добавив к ним доменное имя сайта, и пометить признаком локальности.

```
var hrefs = (from href in Regex.Matches(page, @"href=""[\/\w-\.:]+""").Cast<Match>()
             let url = href.Value.Replace("href=", "").Trim('"')
             let loc = url.StartsWith("/")
             select new
                 {
                     Ref = loc ? $"https://www.susu.ru{url}" : url,
                     IsLocal = loc
                 }
            ).ToList();

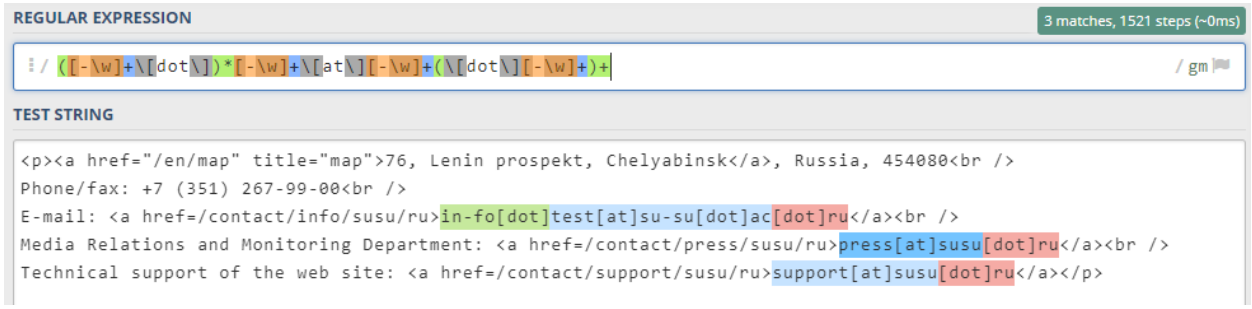
foreach (var href in hrefs)
    Console.WriteLine(href);
```

На выходе LINQ-запроса получим список анонимных объектов со свойствами Ref и IsLocal.

Имея признак локальности IsLocal == true, можно организовать загрузку страниц с адресом из поля Ref.



Для поиска другой информации внутри страницы, например, email-адресов или картинок, также можно написать регулярные выражения.

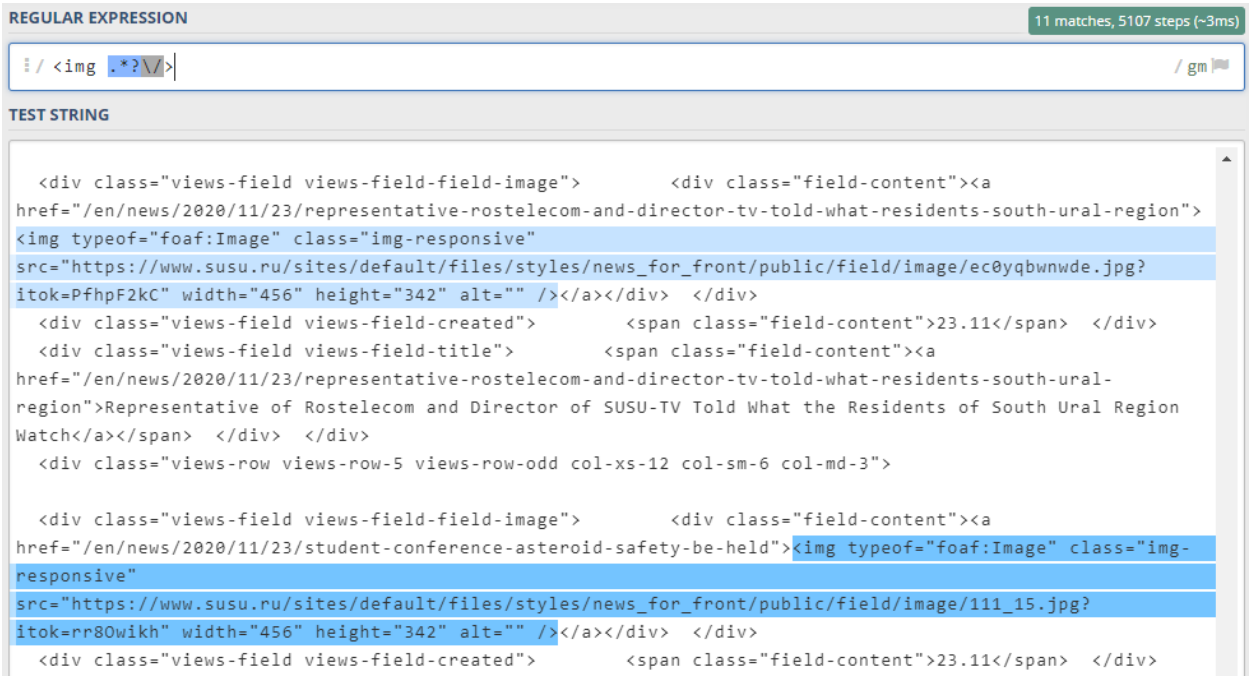


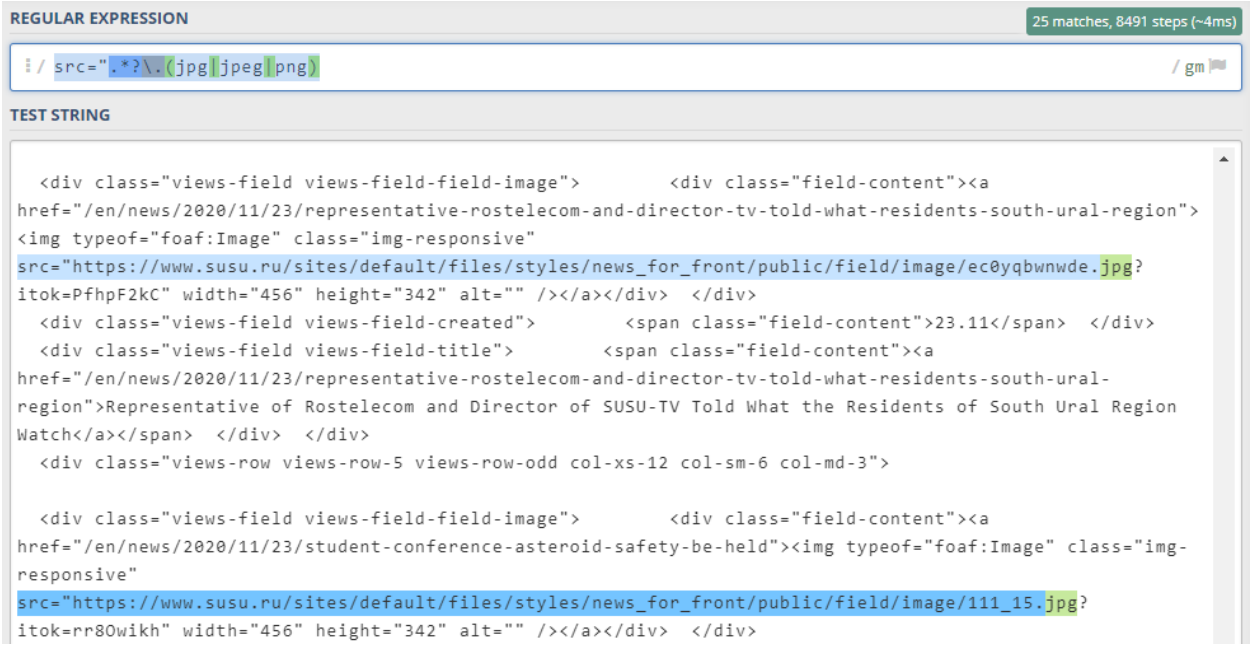
Интересно то, что email на сайте ЮУрГУ имеет замену @ на [at], а точки на [dot].
Шаблон для почты: **([-\w]+\[dot\])*[-\w]+\[at\][-\w]+(\[dot\][-\w]+)+**

Картинка – это тег <img /> с атрибутом ссылки на бинарный контент src. Искать следует либо по тегу, либо по атрибуту:

****
src=".*?\.(jpg|jpeg|png)

Синтаксис регулярных выражений можно найти в сети.





Создадим класс WebScanner, который будет искать на страницах внешние ссылки, передавая их в обработчик события, а также рекурсивно обрабатывать страницы заданного сайта. Ограничим максимальное количество загружаемых страниц 10. Локальными ссылками в рамках сайта будем считать ссылки, принадлежащие одному доменному имени и http-схеме, например, https://www.susu.ru.

```
public class WebScanner : IDisposable
{
    private readonly HashSet<Uri> _procLinks = new HashSet<Uri>();
    private readonly WebClient _webClient = new WebClient();

    private readonly HashSet<string> _ignoreFiles =
        new HashSet<string> {".ico", ".xml"};

    private void OnTargetFound(Uri page, Uri[] links)
    {
        TargetFound?.Invoke(page, links);
    }

    private void Process(string domain, Uri page, int count)...

    public event Action<Uri, Uri[]> TargetFound;

    public void Scan(Uri startPage, int pageCount)
    {
        _procLinks.Clear();

        string domain = $"{startPage.Scheme}://{startPage.Host}";
        Process(domain, startPage, pageCount);
    }

    public void Dispose()
    {
        _webClient.Dispose();
    }
}
```

Сканирование сайта начинается с вызова метода Scan(), в него передается адрес стартовой страницы и максимальное количество обрабатываемых страниц. Само сканирование осуществляется рекурсивным методом Process(). Чтобы дважды не загружать одну и ту же страницу, коллекция _procLinks содержит адреса пройденных страниц. Коллекция _ignoreFiles содержит расширения файлов, которые не нужно загружать.

Событие TargetFound срабатывает в методе Process() после обработки текущей страницы, тип данных Action<Uri, Uri[]> – делегат, позволяющий в обработчик события передавать два параметра: адрес текущей обрабатываемой страницы и массив внешних ссылок на ней.

Локальная переменная string domain = $"{startPage.Scheme}://{startPage.Host}" содержит доменное имя сайта, получаемое из адреса стартовой страницы, используется для определения локальных и внешних ссылок внутри метода Process().

Каждый рекурсивный вызов метода Process() уменьшает на 1 переменную int count. Рекурсия останавливается в случае, если count <= 0, либо больше нет локальных ссылок на страницах.

Рассмотрим код метода Process().

```
private void Process(string domain, Uri page, int count)
{
    if (count <= 0) return;

    if (_procLinks.Contains(page)) return;
    _procLinks.Add(page);

    string html = _webClient.DownloadString(page);

    var hrefs = (from href in Regex.Matches(html, @"href=""[\/\w-\.:]+""").Cast<Match>()
                 let url = href.Value.Replace("href=", "").Trim('"')
                 let loc = url.StartsWith("/")
                 select new
                     {
                         Ref = new Uri(loc ? $"{domain}{url}" : url),
                         IsLocal = loc || url.StartsWith(domain)
                     }
                ).ToList();

    var externals = (from href in hrefs
                     where !href.IsLocal
                     select href.Ref).ToArray();

    if (externals.Length > 0) OnTargetFound(page, externals);

    var locals = (from href in hrefs
                  where href.IsLocal
                  select href.Ref).ToList();

    foreach (var href in locals)
    {
        string fileEx = Path.GetExtension(href.LocalPath).ToLower();
        if (_ignoreFiles.Contains(fileEx)) continue;

        Process(domain, href, --count);
    }
}
```

После того, как мы убедились в том, что еще есть обрабатываемые страницы (count > 0), убедились, что не были на данной странице, загружаем страницу в переменную html. LINQ-запросом hrefs получаем все ссылки на странице, делим их на локальные и внешние, к локальным добавляем доменное имя сайта, чтобы ссылка была полной.

В переменную externals отфильтровываем только внешние ссылки, если они найдены, то инициируем событие, передавая в обработчик найденные ссылки.

В переменную locals отфильтровываем локальные ссылки, и в цикле foreach рекурсивно вызываем Process() для обработки страниц по внутренним ссылкам на них, игнорируя файлы из списка _ignoreFiles.

Осталось создать объект класса WebScanner, добавить обработчик события операцией += и запустить метод сканирования сайта.

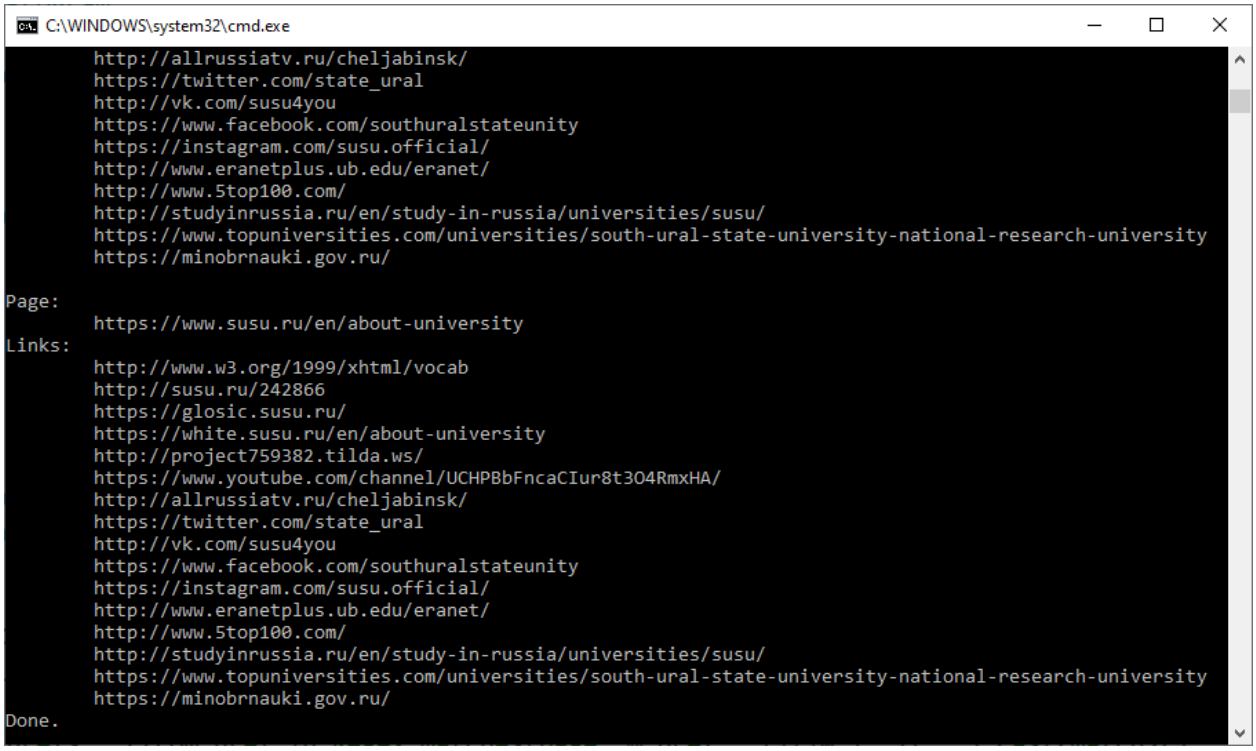
Пример работы сканера сайта.

```
using (WebScanner scanner = new WebScanner())
{
    scanner.TargetFound += (page, links) =>
        {
            Console.WriteLine($"\nPage:\n\t{page}\nLinks:");

            foreach (var link in links)
                Console.WriteLine($"\t{link}");
        };

    scanner.Scan(new Uri("https://www.susu.ru/"), 10);
    Console.WriteLine("Done.");
}
```

Результат работы.



В процессе работы метода Scan() на консоль выводится информация о страницах через обработчик события. Обработчиков может быть несколько, для этого просто добавляем новый обработчик операцией подписки на событие +=.